

Z-SCAPE: Zero-Knowledge Self-Custodial Credential Operation for Privacy-Preserving Asset Protection under Entropy-Source Failure

Mehmet Sabir Kiraz¹ Suleyman Kardas²

¹De Montfort University, Leicester, United Kingdom

²Batman University, Batman, Türkiye

mehmet.kiraz@dmu.ac.uk suleyman.kardas@batman.edu.tr

Abstract

Motivated by the 2026 COLDCARD incident, this paper studies cryptographic asset recovery after self-custodial seed-generation failures. Self-custodial hardware wallets depend on secure entropy sources for seed generation. If an RNG implementation or design failure reduces seed entropy, an adversary may reconstruct wallet signing keys through offline search. Such weaknesses may also be discovered long after wallet creation, placing existing self-custodial assets at risk. To prevent large-scale exploitation after such a failure is identified, a hardware manufacturer or security response team may perform a protective sweep of affected assets into a protected recovery treasury. Asset redistribution then creates a fundamental authentication problem: once the signing key can be reconstructed by both the legitimate owner and an adversary, possession of that key no longer uniquely identifies the legitimate controller.

We propose Z-SCAPE, a zero-knowledge recovery-credential protocol for privacy-preserving asset recovery after seed-generation failures and protective sweeps. Before compromise, the user commits to a recovery credential consisting of a 256-bit recovery secret r generated from an entropy source intended to be independent of the transaction-signing seed, and an RNG-independent personal record P . After an incident, the prover proves knowledge of (P, r) in zero knowledge for the pre-bound wallet identifier W , while binding the proof to the incident-specific protected-asset reference, a fresh verifier nonce, an expiry value, and a fresh recovery destination. The verifier derives the protected-asset reference from authenticated protective-transfer records rather than accepting an arbitrary asset set from the claimant. The protocol enables recovery claims without revealing P , r , or the compromised wallet private keys, while preventing replay, destination substitution, and cross-wallet protected-asset substitution. Z-SCAPE provides concrete integration mechanisms for Bitcoin and Ethereum and enables only assets recorded as protectively transferred from the proved wallet to be returned to the fresh destination bound to an accepted recovery proof.

Keywords: Self-custodial wallets; entropy-source failure; zero knowledge proofs; Bitcoin; Ethereum.

1 Introduction

In a self-custodial wallet, authorization is evaluated by a ledger-specific rule over digital signatures, scripts, or contract state rather than by a custodial identity provider. The security boundary therefore includes entropy generation, deterministic key derivation, secret-key storage, backup, transaction signing, migration, and incident response. Failure of any entropy source upstream of deterministic derivation propagates to every descendant key.

Hierarchical deterministic (HD) wallets instantiate this dependency explicitly. BIP-39 encodes entropy and a checksum as a mnemonic and derives a seed from that mnemonic, while BIP-32 derives a hierarchy of extended private and public keys from the seed [2, 1]. None of mnemonic encoding, cryptographic

hashing, or deterministic key derivation can increase the min-entropy of the originating distribution. The ledger model is chain-specific (e.g., Bitcoin evaluates spending conditions over UTXOs, whereas Ethereum distinguishes externally owned accounts (EOAs) from code-governed contract accounts) but secret-key unpredictability remains necessary wherever an authorization rule ultimately depends on a private key [29].

The 2026 COLDCARD incident is an instance of entropy-source failure at key generation rather than theft of stored key material. Coinkite published an advisory on 30 July 2026, updated on 1 August 2026, identifying affected seed-generation paths on Mk2/Mk3 and later-model firmware [5]. A separate technical analysis attributed the regression to a March 2021 migration from `ckcc.rng_bytes()` to `ngu.random.bytes()` [7]. Independent analysis traced the resulting call graph to MicroPython’s deterministic Yasmarang fallback instead of the intended STM32 hardware RNG [6]. The security loss was therefore caused by the reduced effective support of the seed distribution: runtime isolation, secure storage, and firmware authenticity cannot compensate when the min-entropy available at key generation is not sufficient [23, 24].

1.1 COLDCARD Entropy-Source Failure

Intended entropy pipeline. A hardware-wallet entropy pipeline samples a physical entropy source, applies conditioning, and uses the resulting bit string as input to mnemonic encoding and deterministic key derivation:

$$E_{\text{TRNG}} \rightarrow \text{Condition}(E_{\text{TRNG}}) \rightarrow \text{BIP39} \rightarrow \text{BIP32} \rightarrow \{sk_i, pk_i\}.$$

For a 24-word BIP-39 mnemonic the raw entropy field is 256 bits, and for a 12-word mnemonic it is 128 bits. These bit lengths do not imply equivalent min-entropy unless the source and conditioning pipeline justify the corresponding distributional assumption. Before the regression, Mk2/Mk3 firmware generated 32 bytes by directly calling the STM32 hardware RNG. During the 2021 libNgU migration, the call changed to `ngu.random.bytes(32)` [6, 7]. The failure was caused by code configuration and symbol-linking settings rather than replacement of the cryptographic primitive.

Cross-module configuration and symbol resolution. COLDCARD’s production board configuration contained

```
#define MICROPY_HW_ENABLE_RNG (0)
```

while libNgU used a preprocessor check equivalent to

```
#ifndef MICROPY_HW_ENABLE_RNG
#error "get a HW TRNG plz"
#endif
```

This check tested macro existence rather than its Boolean value. Because the macro was defined as zero, the build-time check succeeded, whereas MicroPython correctly interpreted zero as

disabling the hardware RNG and compiled the Yasmarang fallback. Consequently, `libNgU`'s reference to the global `rng_get()` symbol resolved to the deterministic fallback rather than COLD-CARD's hardware RNG driver [6, 7]. The defect was therefore a software integration error: although the hardware TRNG was present in the compiled binary, build configuration prevented the seed generator from calling it.

Deterministic fallback-state initialization. On affected Mk2/Mk3 firmware, the MicroPython fallback initialized Yasmarang from values of the following form [6]:

$$\begin{aligned} \text{pad} &= \text{UID}_{\text{low32}} \oplus \text{SysTick}, \\ n &= \text{RTC_TR}, \\ d &= \text{RTC_SSR}. \end{aligned}$$

These values are device metadata and timing state, not cryptographic entropy. The MCU UID is fixed for the life of the chip, SysTick is a periodic processor timer, and the RTC fields are correlated with boot/execution timing. After this initialization, no fresh entropy is collected by the fallback: future outputs are deterministic state transitions. Thus, for a fixed UID, timer state and RNG-call history, wallet generation is deterministic [6].

`libNgU` XORed this stream with a second Yasmarang instance initialized from public constants. This composition does not increase min-entropy when both operands are reproducible. In particular,

$$\text{PRNG}_1(S_1) \oplus \text{PRNG}_2(S_2)$$

remains reproducible. XOR preserves unpredictability when at least one independent operand is unpredictable to the adversary, and it provides no entropy amplification when both operands are efficiently computable [6, 39].

Output length versus source min-entropy. The wallet subsequently hashed the 32-byte RNG output. A cryptographic hash may serve as a conditioning function under defined assumptions, but it cannot create entropy or increase the number of possible secrets. If the vulnerable generator can produce at most N candidate outputs, then

$$|\{\text{SHA256d}(x) : x \in \mathcal{X}\}| \leq |\mathcal{X}| = N.$$

The BIP-39 checksum and BIP-32 derivation likewise cannot restore entropy missing from the source distribution. Output length and effective min-entropy are distinct quantities [23].

For Mk2/Mk3, Block gives several attacker-knowledge scenarios rather than a single entropy number. If UID, timers, and call history are known, the candidate count collapses to 2^0 : the output is deterministic. With known UID but unknown SysTick under a normal cold-boot model, the maximum SysTick enumeration is only about $80,000 \approx 2^{16.29}$. A theoretical upper bound assuming independent timer fields is

$$80,000 \times 86,400 \times 256 \approx 2^{40.7}.$$

The value $2^{40.7}$ is an upper bound on a modeled generator-state family, not the bit length of a secp256k1 secret key. The curve's scalar field is not changed, and the vulnerability restricts the support of the distribution from which wallet keys are deterministically derived [6].

Limitations of online health tests. The deterministic fallback did not necessarily produce repeated or low-diversity byte strings. Block notes that both the adjacent-value check and the requirement for more than four distinct bytes can be satisfied by Yasmarang [6]. Such tests detect a restricted class of catastrophic failures but do not establish unpredictability, source min-entropy, or reachability of the intended entropy source.

Later models and competing entropy estimates. Mk4, Q and Mk5 introduced secure-element material at boot. Coinkite estimates that affected later-model seeds had an effective search space of about 72 bits rather than the intended 128-bit security target [5, 7]. Block's analysis is more conservative. It observes that the secure-element values are hashed, but only the first four digest bytes are converted to an integer and passed to `ngu.random.reseed()`, and that function replaces only one 32-bit Yasmarang state word. For a fixed fallback state and call history, Block therefore bounds the number of securely distinguished streams by 2^{32} , while a deliberately loose hidden-timer ceiling can rise toward $2^{73.3}$. Block explicitly warns that the latter is not equivalent to 73 bits of cryptographic security because the timers are correlated and potentially reconstructable [6]. Accordingly, the vendor estimate and Block's independent bound represent different attacker models and should not be interpreted as a single entropy estimate.

Offline key-enumeration attack. Bitcoin's public ledger supplies an offline candidate-validation oracle: an adversary need not query, infect, or physically access the target device. An attack consistent with the published root-cause analysis proceeds as follows:

1. enumerate candidate fallback states (UID, SysTick, RTC, T), where T represents feasible prior RNG-call histories,
2. reproduce the corresponding 32-byte `ngu.random` stream and the wallet's subsequent hash/seed-generation steps,
3. derive BIP-39/BIP-32 wallet keys and standard address paths for each candidate,
4. query an indexed Bitcoin ledger for a match with each candidate address or public key,
5. when a candidate address matches a funded output, derive the corresponding private key(s), construct sweep transactions, and broadcast them.

Figure 1 summarizes the offline key-enumeration pipeline.

The target device may remain completely offline throughout the attack. A public address, extended public key, or revealed public key permits rejection of incorrect candidates and confirmation of a matching derived state [6]. For the initial 30 July sweep, CoinDesk, citing Galaxy Research, reported 1,082.65 BTC transferred from 1,196 wallets within 41 minutes [8]. Subsequent reporting identified additional waves, and by 3 August 2026, a possible fourth sweep brought the reported total to about 1,816 BTC from more than 5,200 addresses, approximately \$114 million at the reported prices [9]. The transaction pattern is consistent with systematic candidate enumeration, although public ledger evidence alone does not determine the attack implementation.

Irreversibility for previously generated seeds. Air-gapping constrains communication, secure elements constrain key extraction, and signed firmware constrains unauthorized code execution. None of these measures increases the uncertainty of a seed that was already generated from an enumerable state space. Re-importing the same seed into another device likewise preserves the compromised root. A firmware correction affects only subsequently generated seeds. The required remediation is therefore to generate a fresh seed from an independent entropy source and migrate the affected funds [5]. The vendor advisory lists Mk2/Mk3 seeds generated on versions 4.0.1–4.1.9 as affected, while Block traces the vulnerable code path to v4.0.0. Coinkite also identifies Mk4/Mk5 seeds generated before standard 5.6.0 or Edge 6.6.0X, and Q seeds generated before standard 1.5.0Q or Edge 6.6.0QX, as affected [5, 6].

The incident establishes key-generation integrity as part of the trusted computing base. Security analysis must model the

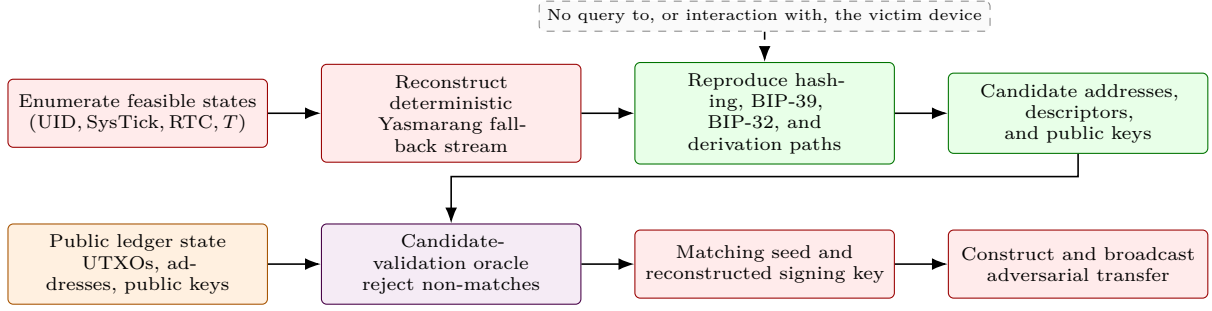


Figure 1: Offline attack flow caused by the COLDCARD entropy failure. The attacker searches possible fallback states, derives candidate keys, and verifies them against public ledger data. A matching candidate reveals the signing key without contacting the target device.

firmware version, compiled binary, entropy-source call graph, conditioning function, and externally verifiable evidence that the intended entropy source was reached [23, 24, 6]. Where a wallet supports externally supplied entropy, the design should also provide a mechanism for the user to verify that the intended contribution affected wallet generation. Where the derivation architecture makes this independently checkable, a derived extended public key may form part of that verification; otherwise a dedicated verifiable entropy-contribution mechanism is required.

Let an adversary reconstruct the seed and signing keys of a vulnerable wallet. Both the pre-compromise controller and the adversary can then generate accepting signatures. Hence

$$\text{VerifySig}(pk, m, \sigma) = 1$$

establishes only possession of a valid signing witness at verification time. It does not establish prior control, pre-compromise enrolment, or legal title.

To prevent mass exploitation upon discovering an RNG defect, hardware manufacturers or security response teams can protect vulnerable funds in advance by sweeping exposed outputs into a protected recovery treasury. Redistributing these secured assets, however, creates a fundamental authentication problem: because the compromised signing keys are computable by any party, possession of the private key no longer proves legitimate ownership. Subsequent asset return requires a recovery-authentication mechanism that operates independently of the compromised key.

We address the following problem: how to bind the wallet to two recovery factors with different failure modes while the native wallet authority is still trustworthy. The first is a 256-bit recovery secret r generated through an entropy source intended to be independent of the transaction-signing seed. The second is a mandatory personal record P , containing the home address and at least three security-question answers, whose values do not depend on the wallet RNG. Rather than acting as conventional knowledge-based authentication, P diversifies the failure mode of the recovery witness. When r maintains its intended conditional min-entropy, recovery security is dominated by r . If a hidden defect later shows that the RNG used for r shared a vulnerable hardware component, software library, entropy-generation path, or implementation dependency with the signing-key RNG, the entropy of r may be much lower than its 256-bit representation suggests. In that case, the components of P that remain unknown to the adversary form an RNG-independent source of residual conditional uncertainty. Z-SCAPE publishes no user name, email address, home address, selected questions, or answers. If the wallet is not reliably linked to a particular person, the adversary may face a broader distribution of plausible personal records than for a known controller. This can affect the conditional distribution of P , but it is not counted as a separate additive security term. This is not treated as a guaranteed security margin: blockchain analysis, KYC records, public disclosures, breached data, or other auxiliary information may reveal the wallet controller and reduce or eliminate that benefit.

Security therefore depends on the joint conditional distribution of (P, r) given the adversary’s complete view, evaluated jointly rather than as additive entropy contributions. After an incident, the prover proves knowledge of the pre-enrolled opening (P, r) in zero knowledge (ZK) and binds the proof to a fresh recovery destination. The proof establishes knowledge of the recovery credential, not legal ownership.

1.2 Contributions

Z-SCAPE addresses a distinct recovery setting: a latent seed-generation failure is discovered after enrolment, the native wallet signing authority has become non-unique because the signing key is reconstructable, and affected assets may first need to be moved into a protected recovery treasury before redistribution. Prior work on signature lifting and post-quantum Bitcoin rescue uses knowledge asymmetry that can remain after a classical signature key becomes unsafe [10, 11, 12]. Z-SCAPE instead pre-enrolls a recovery witness whose security is intended to be independent of the transaction-signing seed-generation path and uses that witness to authenticate privacy-preserving redistribution after a protective transfer. The motivating problem is not limited to known RNG failures. An RNG considered secure at deployment may later be found vulnerable because of implementation errors, design weaknesses, poor entropy sources, hardware faults, software bugs, or other failures that reduce the effective randomness of generated seeds. Such weaknesses may remain undetected for years, meaning that even long-held self-custodial assets may become vulnerable if their signing keys can later be reconstructed through offline search. In this case, possession of the signing key no longer provides unique evidence of wallet ownership, because both the legitimate owner and an attacker may be able to derive the same key.

Z-SCAPE addresses this problem through an emergency recovery protocol in which a hardware manufacturer or security team can perform a protective sweep of affected assets into a protected recovery treasury before malicious exploitation, and later redistribute the protected assets using ZK proofs. The protocol uses a recovery credential consisting of a 256-bit recovery secret r generated through an entropy path intended to be independent of the signing seed, and an RNG-independent personal record P , such as the enrolment-time home address and answers to private personal-history questions. If r is affected by the same entropy failure as the signing key, recovery can still rely on the attacker’s remaining uncertainty about P .

Z-SCAPE defines a ZK proof-of-knowledge relation that proves knowledge of (P, r) while binding each recovery claim to the wallet identifier, incident, authenticated protected-asset reference, verifier nonce, expiry value, and fresh destination address. The protected-asset reference is derived by the verifier from authenticated protective-transfer records rather than from an arbitrary asset set supplied by the claimant. Consequently, a valid recovery credential for one wallet cannot authorize assets recorded as having been protectively transferred from another

wallet. The construction prevents replay, destination-address substitution, and cross-wallet protected-asset substitution without revealing P , r , or the compromised wallet private keys. The framework also provides crypto-agility by separating the stable wallet identifier W from the proof system and commitment-hash configuration, allowing these cryptographic components to be upgraded without changing the wallet identifier. We define concrete integration mechanisms for Bitcoin using an `OP_RETURN`-based data-carrier record that binds the recovery commitment to the wallet, and for Ethereum using registry-based verification with either on-chain or threshold-authorized off-chain verification. We further analyse the main security, privacy, and governance requirements of the recovery process [26, 27, 28].

1.3 Roadmap

Section 2 reviews entropy-source standards, hardware-wallet security and recovery designs, blockchain account-recovery mechanisms, and the ZK tools used by Z-SCAPE. Section 3 specifies the chain-independent Z-SCAPE protocol, including the recovery credential, enrolment, ZK recovery claim, proof system profiles, and authenticated credential updates. Section 4 presents the Bitcoin and Ethereum instantiations. Section 5 describes recovery-credential custody, pre-authorized protective transfer, and claim verification. Section 6 defines the implementation and evaluation plan. Section 7 analyses the adversarial model and the main security properties. Finally, Section 8 discusses deployment, privacy, and governance considerations and concludes the paper.

2 Related Work

Entropy-source evaluation and deterministic random-bit generation are addressed separately in NIST SP 800-90B and SP 800-90C [23, 24]. These standards emphasize source modeling, min-entropy estimation, health testing, and correct integration of the intended entropy path. Hardware wallets apply these concerns in a setting where a dedicated device generates or stores private-key material and performs transaction signing while limiting exposure to the host computer. A 2025 large-scale study evaluated 17 hardware-wallet models from 11 vendors and collected 3.4 million recovery phrases and 3.4 million ECDSA signatures, and the authors reported no significant cryptographic weaknesses in the tested devices, while highlighting the difficulty of reproducible testing of physical wallets [13]. Complementary work shows that the signing device itself remains part of the trusted computing base: Scott and Andersen demonstrated a functional backdoored Bitcoin hardware wallet that leaked its seed through otherwise valid ECDSA signatures observable on the public blockchain [14]. The 2026 COLDCARD incident illustrates a different failure mode in the same trust boundary, where the implemented seed-generation path reached a deterministic software fallback instead of the intended hardware entropy source [5, 7, 6].

Representative self-custody hardware wallets combine Secure Elements, open-source firmware, air-gapped or offline signing, multiple entropy inputs, and backup mechanisms through distinct architectural choices. Ledger and Trezor document Secure-Element-based protection and recovery mechanisms, COLDCARD documents dual Secure Elements, air-gapped operation, and user-supplied dice entropy, BitBox02 documents a dual-chip design with multiple entropy inputs, Blockstream Jade documents offline signing and its blind-oracle “virtual secure element” design, and Keystone and SafePal document Secure-Element and air-gapped designs [15, 16, 17, 18, 19, 20, 21, 22]. These mechanisms are intended to strengthen seed generation, private-key isolation, resistance to physical extraction, transaction confirmation, and recovery from device loss. Their normal recovery mechanisms, however, restore the same signing seed or shares derived from that seed. Splitting a backup with Shamir secret sharing [37] can reduce loss and theft risk, but it does

not increase the entropy of an already generated master secret. Similarly, a Secure Element or air gap cannot later increase the uncertainty of a seed drawn from a reduced candidate space. Designs that combine independent entropy sources or allow external dice entropy reduce the probability of a common-mode generation failure, but they do not by themselves provide independent recovery evidence that distinguishes the pre-compromise controller from an adversary who has reconstructed the same signing key. Z-SCAPE addresses this separate problem by enrolling an independent recovery secret and private recovery record before compromise and later proving knowledge of that opening without relying on the compromised signing key.

BIP-39 and BIP-32 standardize mnemonic encoding and hierarchical deterministic derivation, respectively [2, 1], and they do not recover security lost when the seed has insufficient entropy. Bitcoin output descriptors describe collections of output scripts and provide a useful basis for defining the wallet derivation domain [3]. Z-SCAPE does not assume that a descriptor has a unique canonical textual representation supplied by BIP-380. Instead, its Bitcoin adapter defines a protocol-specific byte encoding and verifies descriptor membership from an explicit derivation witness. On Ethereum, ERC-1271 and ERC-4337 support programmable signature validation and account logic [32, 33]. Such mechanisms can implement recovery through a threshold authorization scheme or secret-sharing mechanism when configured before compromise. These approaches change or extend the authorization path for future state transitions, whereas Z-SCAPE is designed for the case in which a key-dependent authorization rule has already ceased to identify a unique controller.

Related post-compromise rescue mechanisms have been studied in the post-quantum setting. Sattath and Wyborski’s signature-lifting construction shows how additional one-way structure in key derivation can preserve a knowledge asymmetry after the deployed signature scheme becomes vulnerable to a quantum adversary [10]. BIP-361 similarly discusses quantum-safe rescue protocols that distinguish an authentic holder through knowledge not available from a quantum-recovered signing key [11]. Osuntokun demonstrated a zk-STARK prototype for BIP-32/BIP-86 recovery based on proving knowledge of upstream derivation material [12]. These mechanisms are closely related at the level of recovery authentication, but they rely on an upstream secret or derivation relation that remains unavailable to the attacker. In the entropy-failure setting considered here, the signing seed itself may be enumerable, so derivation material descended from that seed cannot be assumed to provide an independent recovery witness. Z-SCAPE therefore pre-enrolls a separate recovery credential and additionally models the protective-transfer and treasury-redistribution phase.

ZK arguments allow a prover to show knowledge of a witness satisfying a relation without disclosing that witness. Non-interactive constructions are typically obtained via the Fiat-Shamir heuristic or its algebraic variants [38]. PLONK provides a universal zk-SNARK construction with succinct verification and an updatable universal structured reference string [26]. ZK-STARKs provide transparent proof systems whose security can be based on hash-function assumptions and avoid a trusted setup [27]. Hash choice also affects proving cost: SHA-256 can be represented in general-purpose proof systems, but its bit-oriented structure is expensive in arithmetic circuits, whereas Poseidon was designed for arithmetic over prime fields and can reduce the constraint cost of hash-preimage statements [28]. Z-SCAPE therefore separates the long-lived wallet identifier and recovery credential from replaceable proof system and commitment-hash profiles. Unlike social-recovery and threshold-signing schemes that alter spending authorization rules, Z-SCAPE leaves the native wallet rule unchanged and provides an independently recorded recovery proof for off-chain claim verification or for a recovery module explicitly programmed to consume such evidence. The mandatory personal record is an additional factor

whose values do not depend on the wallet RNG. The personal fields are never published or disclosed during a normal claim, and the ZK relation proves only that the prover knows the exact pre-enrolled values together with the recovery secret.

NIST SP 800-63B-4 rejects knowledge-based authentication and security questions as authentication mechanisms because human-derived answers can have small candidate spaces and may be obtainable from public or breached information [25]. Z-SCAPE avoids using P for direct authentication: the values are never presented to a verifier for comparison, and are committed before compromise jointly with r . The purpose of P is failure-mode diversification: if the RNG for r shares a hidden defect with the signing key, any unknown components of P provide residual conditional uncertainty for (P, r) . Because the public state contains no personal fields, lack of a reliable wallet-to-person link can broaden the distribution of plausible values of P . Z-SCAPE evaluates this effect only through the joint conditional distribution of (P, r) given the adversary’s view; it does not assign separate additive entropy to an unknown identity, and it explicitly accounts for identity linkage from blockchain analysis or KYC records.

3 Our ZK Recovery-Credential Protocol

The cryptographic design separates three objects: a stable wallet context, a recovery credential, and ledger-specific transaction mechanics. The credential contains both a recovery secret generated through an entropy source intended to be independent of the signing seed and a mandatory canonically encoded personal record. A prover later proves knowledge of a valid opening of the active commitment in ZK and binds the proof to a fresh recovery destination through the proof’s public inputs.

3.1 Entities, profiles, and notation

The protocol involves a wallet controller $\mathcal{U}$, an authenticated append-only ledger $\mathcal{L}$, a claim verifier or treasury operator $\mathcal{T}$, and an adversary $\mathcal{A}$. Let

$$\Pi_z = (\text{Setup}_z, \text{Prove}_z, \text{Verify}_z)$$

denote an allowed ZK argument-of-knowledge profile identified by a profile string z . The profile may be instantiated by a PLONK-family zk-SNARK or by a zk-STARK [26, 27]. The protocol requires completeness, knowledge soundness, and ZK for the relations defined below.

Let $H_{\text{id}} = \text{SHA256}$ be the fixed wallet-identifier hash and let $H_{\text{ctx}} = \text{SHA256}$ be the domain-separated claim-context hash. Commitment hashes are selected from a versioned set $\mathcal{H}_{\text{cred}}$, and the profiles specified here are Poseidon over a proof system-compatible prime field and SHA-256 [28]. Every commitment-hash profile used by the Bitcoin data-carrier adapter must define a canonical 32-byte serialization of C ; a future profile with a different output length requires a new record version rather than reinterpretation of the existing format. The proof system identifier z is not committed into the long-lived credential. A verifier instead checks that the circuit/version associated with z implements the credential relation for the hash profile named by the active record.

Let:

- D denote the protocol-specific canonical wallet context containing the Z-SCAPE namespace, a wallet-context encoding version fixed for the lifetime of W , network identifier, account type, descriptor or account identifier, and credential domain (the derivation-branch range or account set covered by the credential),
- $W = H_{\text{id}}(\text{Z-SCAPE/wallet/v3} \parallel \text{Encode}(D))$ denote the stable identifier of the original wallet being protected. It identifies the wallet whose assets may later require protective

recovery and is distinct from the fresh recovery destination D_{dest} ,

- $P = (a_{\text{home}}, (q_1, a_1), \dots, (q_m, a_m))$ denote the mandatory canonical private recovery record, where a_{home} is the enrolment-time home address and $m \geq 3$ security-question answers are recorded,
- $r \leftarrow \{0, 1\}^{256}$ denote a recovery secret generated from an entropy source intended to be independent of the transaction-signing seed,
- pol denote the committed policy core containing credential version, sequence number, recovery mode, ledger-adapter metadata, and where protective transfer is enabled, the authorized rescue entity or threshold committee, permitted treasury configuration, trigger class, and asset/chain scope, but excluding the proof system identifier,
- h denote the active commitment-hash profile,
- incident denote a unique incident identifier within the Z-SCAPE namespace,
- assetRef denote an authenticated reference to the exact set of assets protectively transferred from wallet W under incident , including the chain-specific source references and treasury-transfer references,
- D_{dest} denote a canonical, chain-separated fresh asset-return destination,
- $n \leftarrow \{0, 1\}^{256}$ denote a fresh verifier nonce,
- expiry denote the claim-expiry value under the ledger adapter’s canonical time or block-height semantics,
- γ denote the domain-separated digest of the complete public claim context,
- C denote the active credential commitment, and
- α denote the Z-SCAPE ledger record together with the adapter-specific inclusion and finality evidence.

$$\text{assetRef} = H_{\text{id}}(\text{Z-SCAPE/protected-assets/v1} \parallel$$

$$\text{Encode}(\text{incident}, W, \text{sources}, \text{treasuryRefs}, \text{assetSet})). \quad (1)$$

Here, sources , treasuryRefs , and assetSet are derived by the recovery operator from authenticated ledger and protective-transfer records and are not selected by the claimant.

The encoding of D , P , and pol is deterministic, prefix-free, versioned, and domain-separated. The wallet-context encoding version is fixed when W is created and is not changed by software upgrades, commitment-hash migration, or proof system migration, and such changes therefore do not alter the stable wallet identifier. A Bitcoin mainnet descriptor, an Ethereum mainnet EOA, an EIP-7702 delegated EOA, an Ethereum contract account, and an identical hexadecimal address on another EVM-compatible network are distinct protocol subjects. For P , the profile fixes Unicode normalization, case handling, whitespace, date and address formatting, question identifiers, answer encoding, and missing-value rules. The user must preserve enough information to reconstruct the exact canonical record.

Define $\text{WF}_P(P) = 1$ if and only if P contains a non-empty canonical enrolment-time home address, at least three distinct approved question identifiers, a normalized answer for each selected question, and satisfies all versioned encoding rules. Initial enrolment computes a credential commitment only when $\text{WF}_P(P) = 1$. At least two of the $m \geq 3$ security-question answers should be selected from personal-history questions (e.g.,

childhood events, first experiences) or other memory-based questions that the enrollee reasonably considers non-public rather than from public-record attributes. This is an enrolment-policy condition rather than a cryptographically verifiable property: whether an answer is in fact obtainable from public records, breached data, or other auxiliary sources cannot be established by the ZK circuit and is therefore not part of WF_P .

3.2 Credential hash encoding

For the SHA-256 configuration, H_h is computed over the exact canonical byte string. For the Poseidon configuration over a prime field $\mathbb{F}_q$, Z-SCAPE does not attempt to inject an arbitrary byte string into one field element. Let

$$b = \lfloor \log_2 q \rfloor - 1.$$

The byte representation is first prefixed by its bit length and a domain tag, then split into b -bit chunks. Each chunk is interpreted as a non-negative integer strictly below $2^b < q$ and therefore maps injectively to one element of $\mathbb{F}_q$. The resulting vector, including the explicit length element, is absorbed by the versioned Poseidon sponge. The field modulus, rate, capacity, round numbers, MDS matrix, round constants, byte order, chunk order, and padding rule are fixed by the profile identifier. This construction is injective at the byte-to-field-vector layer, and the final digest remains a cryptographic hash output and is not itself an injective representation of arbitrary input strings. The operator $ZKEncode$ used in the credential commitment denotes this proof system-specific byte-to-field encoding of P , while $Encode$ denotes the general deterministic byte encoding used elsewhere.

3.3 Recovery credential

The wallet samples

$$r \leftarrow \{0, 1\}^{256}$$

from an entropy source intended to be independent of the transaction-signing seed. The recovery secret should not reuse the wallet's seed-generation RNG path and should, where practical, be generated by a separately implemented or externally supplied entropy source. A deployment may also combine independently obtained entropy inputs through a domain-separated cryptographic extractor before deriving r . These measures reduce common-mode risk, but the independence and entropy of r remain implementation assumptions. A hidden defect may later reveal that the source used for r shared a hardware component, software library, entropy-generation path, or other dependency with the RNG used for the signing seed. Consequently, the fact that r is represented as a 256-bit string does not imply that it necessarily maintains 256 bits of conditional min-entropy after such a defect is discovered. If genuinely independent high-entropy randomness is available at wallet creation, it should also be used as a preventive control by contributing to the signing-seed generation process. Z-SCAPE is not intended to replace sound RNG design, independent entropy mixing, or verification that the intended entropy path was actually used. Its purpose is different: it preserves a separate recovery witness that remains available if a seed-generation failure is nevertheless discovered after enrolment. Where practical, the entropy used for seed strengthening and the entropy used to derive r should come from separate failure domains; if they share an underlying source or implementation dependency, that common-mode risk is included in the Z-SCAPE threat model.

The user also constructs the mandatory record

$$P = (a_{\text{home}}, (q_1, a_1), \dots, (q_m, a_m)), \quad m \geq 3,$$

where a_{home} is the residential address at enrolment and each q_i is a versioned question identifier with normalized answer a_i . Unlike r , these values are human-derived and do not depend on

the wallet RNG. Z-SCAPE does not treat P as a standalone authenticator and does not infer a fixed amount of entropy from names, addresses, or security-question answers. Its purpose is to introduce an RNG-independent factor into the committed recovery witness. If the entropy of r is later found to be substantially lower than assumed, an attacker testing candidate openings must also recover or guess the components of P that are not already determined by public records, breached data, social knowledge, or other auxiliary information.

The public Z-SCAPE state does not contain the user's name, email address, home address, selected question identifiers, or answers. Therefore, when a wallet cannot be reliably linked to a particular person, an attacker does not start with a known personal profile and must additionally identify candidate wallet holders before efficiently enumerating P . This can increase the conditional uncertainty of the complete opening. Z-SCAPE does not assume that such unlinkability always holds: blockchain analysis, KYC records, exchange records, public disclosures, device records, or data breaches may reduce or eliminate this advantage.

Z-SCAPE requires both P and r for every recovery proof because the system cannot know at enrolment which source, if any, may later prove defective. When r maintains high conditional min-entropy, recovery security is dominated by r and P is not relied upon for additional cryptographic protection. In the degraded-entropy case, any residual uncertainty in P may increase the cost of offline recovery because its values are not generated by the wallet RNG. Security therefore depends on the joint conditional distribution of (P, r) given the adversary's view, not on adding nominal entropy values for the two factors. In particular, Z-SCAPE does not claim that three security questions provide a fixed number of security bits or that they compensate for a completely predictable r when P is also known or efficiently enumerable. This role is distinct from conventional knowledge-based authentication: the verifier never accepts P by itself and does not learn its values during a normal claim.

3.4 Illustrative quantitative analysis of the personal record

The preceding definition intentionally does not assign a fixed entropy value to a home address or to an individual security-question answer. Human-derived values are non-uniform, may be correlated, and may become partially determined by public records, breached databases, OSINT, social knowledge, or knowledge of other answers. Nevertheless, it is useful to show how a deployment can translate measured or conservatively estimated conditional guessing probabilities into an opening-search bound.

The numerical values below are deliberately synthetic. They are not empirical measurements, are not normative Z-SCAPE parameters, and must not be interpreted as universal entropy estimates for the corresponding questions. In particular, the minimum policy of an enrolment-time address plus three questions does not inherit the numerical bound of the twenty-component example below.

Write the personal record as an ordered sequence

$$P = (X_1, \dots, X_\ell),$$

where the components include the canonical enrolment-time home address and the selected question-answer pairs. Let $V_A = v$ be the complete adversarial view, including any wallet-to-person linkage that is actually available. For each component X_j , define s_j by the pointwise conditional guessing bound

$$\max_x \Pr[X_j = x \mid X_{<j} = x_{<j}, V_A = v] \leq 2^{-s_j}, \quad (2)$$

for every feasible preceding history, where $X_{<j} = (X_1, \dots, X_{j-1})$. Thus s_j is not the standalone entropy of an answer. It is the remaining uncertainty after all preceding personal fields and all auxiliary information in the adversarial view have

Table 1: Illustrative conditional guessing bounds for a 20-component personal record. Values are not empirical measurements or normative Z-SCAPE parameters.

Personal-record component	s_j (bits)
Enrolment-time home address	14
Primary-school name	6
First pet’s name	5
Mother’s maiden surname	7
First teacher’s surname	6
Closest childhood friend’s name	6
Childhood street name	7
First paid employer	7
First manager’s surname	6
First vehicle make and model	5
First foreign city visited	4
Childhood nickname	5
Favourite childhood teacher	5
First major personal purchase	4
Childhood holiday location	5
Childhood friend’s surname	5
First mobile-phone model	4
Private family nickname	5
First sports club or team	4
First computer make or model	5
An illustrative total	111

already been taken into account. This conditioning is important. For example, knowledge of a childhood address may substantially reduce the uncertainty of a primary-school name, and knowledge of a person’s age, location, profession, or identity may reduce the uncertainty of a first employer. Such information is represented inside V_A and must therefore reduce the calibrated s_j values where appropriate rather than being treated as an independent additive factor. Table 1 gives one hypothetical calibration for a deployment using an enrolment-time address and nineteen private personal-history questions. The values are conditional bounds chosen only to illustrate the calculation.

Under the pointwise sequential bounds in Equation (2), the probability of any particular complete personal record satisfies

$$\Pr[P = p \mid V_A = v] \leq 2^{-\sum_{j=1}^{\ell} s_j}. \quad (3)$$

For the values in Table 1, this becomes 2^{-111} . This is a property of the assumed conditional distributions, not a measured security level of security questions. The selected question identifiers q_i are also absent from the public Z-SCAPE state. If their uncertainty is relevant, it must already be reflected in the joint distribution of P and must not be counted a second time. The same rule applies to wallet-to-person unlinkability. An unknown controller can make constructing high-probability guesses operationally harder and can change the conditional distribution of P , but identity uncertainty is not an independent cryptographic opening factor because the committed opening contains (P, r) and not a separate identity variable. Any benefit from unlinkability is therefore captured only through the calibrated conditional bounds for P under the actual adversarial view. If r maintains a pointwise conditional guessing bound of $t = 32$ bits after the entropy-source failure, then the 111-bit personal-record example would give a combined modeled opening bound of

$$111 + 32 = 143$$

bits under the sequential assumptions formalized in Section 7.3. If instead r is completely predictable, the same example gives only the 111-bit modeled bound, which is below a 128-bit target. These examples are illustrative only: the values s_j and t must be justified for the deployed question set, available auxiliary information, and entropy-failure scenario.

3.5 Initial enrolment

The protected wallet context D , and therefore its stable identifier W , must already exist before Z-SCAPE enrolment. Thus, the

recovery credential is committed after creation of the wallet being protected, but before compromise of its native signing authority. For active hash profile h , define

$$C = H_h(\text{Z-SCAPE/credential/v6} \parallel W \parallel \text{ZKEncode}(P) \parallel r \parallel H_{\text{id}}(\text{Encode}(\text{pol}))). \quad (4)$$

The enrolment records the protocol version, action type, hash-profile identifier h , sequence number, stable wallet identifier W , and serialized commitment C . It does not contain P , r , or the proof system identifier. Equation (4) is a hash-based commitment. Second-preimage and collision resistance support its binding property, but collision resistance alone is not a hiding assumption when components of the opening have low entropy. Under normal operation, hiding of the personal record relies primarily on the infeasibility of enumerating the joint opening because r maintains high conditional min-entropy. If r later proves weak, privacy and offline-guessing resistance depend on the joint conditional distribution of (P, r) given the adversary’s auxiliary information and, where pseudorandomness of the digest is claimed, on an explicit hash-model assumption such as a random-oracle or comparable pseudorandomness assumption. The ledger adapter authenticates initial enrolment using the wallet’s native authorization rule and records sufficient evidence to verify that the enrolling authority corresponded to D at ledger time τ_e .

Each ledger adapter defines a canonical rule for selecting or enforcing a unique initial Z-SCAPE enrolment for wallet context D . For Bitcoin, the initial commitment is selected from finalized eligible Z-SCAPE records under the order defined in Section 4.1, and for an Ethereum registry, uniqueness is enforced by the registry’s initial-state transition. A later native-wallet signature alone is not sufficient to replace the active commitment. The purpose of this pre-compromise enrolment is to establish the recovery credential while the native wallet authority is still trustworthy. If a new recovery credential could be enrolled only after the signing key became reconstructable, both the legitimate controller and an adversary knowing the same reconstructed key could authenticate a different recovery credential. By recording the commitment C for W before such a compromise, later recovery can require knowledge of the credential that was already associated with that wallet before the native signing authority became ambiguous.

3.6 ZK Recovery Claim

The wallet-to-credential binding is established at enrolment and is therefore pre-bound rather than late-bound. Specifically, the commitment C is bound to the stable identifier W of the wallet being protected while the native wallet authority is still trusted. What is intentionally bound later, at claim time, is the incident-specific recovery context. The pre-enrolled commitment is not intended to identify a future incident, protected-asset set, or recovery destination. After an incident, the recovery operator independently constructs an authenticated protected-asset record for each affected W , identifying only the assets actually transferred from that wallet into the recovery treasury.

During recovery, the relation $\mathcal{R}_{\text{claim}}$ requires the prover to open the active pre-enrolled commitment for W and binds the proof to incident, the corresponding `assetRef`, D_{dest} , n , and expiry. The claimant does not select an arbitrary protected-asset set: the verifier resolves `assetRef` independently from the authenticated protective-transfer records for $(\text{incident}, W)$. Consequently, a valid recovery credential for one wallet cannot authorize the return of assets recorded as having been swept from another wallet. Define the domain-separated public claim-context digest

$$\gamma = H_{\text{ctx}}(\text{Z-SCAPE/claim-context/v2} \parallel \text{Encode}(W, C, h, \text{pol}, \text{incident}, \text{assetRef}, D_{\text{dest}}, n, \text{expiry})). \quad (5)$$

Define the public claim instance

$$x_{\text{claim}} = (W, C, h, \text{pol}, \text{incident}, \text{assetRef}, D_{\text{dest}}, n, \text{expiry}, \gamma)$$

and private witness

$$w_{\text{claim}} = (P, r).$$

The relation $\mathcal{R}_{\text{claim}}$ accepts exactly when the private record is well formed, the witness opens the active commitment, and the complete public claim context is constrained by Equation (5):

$$\text{WF}_P(P) = 1,$$

$$C = H_h(\text{Z-SCAPE/credential/v6} \parallel W \parallel \text{ZKEncode}(P) \parallel r \parallel H_{\text{id}}(\text{Encode}(\text{pol}))),$$

$$\gamma = H_{\text{ctx}}(\text{Z-SCAPE/claim-context/v2} \parallel \text{Encode}(W, C, h, \text{pol}, \text{incident}, \text{assetRef}, D_{\text{dest}}, n, \text{expiry})). \quad (6)$$

A valid proof system instantiation must constrain every component of the canonical public claim context through the final equality in Equation (6). The incident identifier, protected-asset reference, destination, nonce, and expiry are therefore part of the proved relation rather than verifier-side metadata. A circuit or AIR that omits any field from the context encoding or leaves the context-digest equality unconstrained is not a valid Z-SCAPE claim implementation. Figure 2 illustrates the complete enrolment and recovery proof flow. The prover computes

$$\pi_{\text{claim}} \leftarrow \text{Prove}_z(pp_z, x_{\text{claim}}; P, r)$$

and sends $(D_{\text{dest}}, \pi_{\text{claim}})$ together with the enrolment-record reference and public policy information. The verifier accepts only if C is the active authenticated commitment for W , the selected circuit implements Equation (6) for profile h , and

$$\text{Verify}_z(v_{k_z}, x_{\text{claim}}, \pi_{\text{claim}}) = 1.$$

Because Equation (6) explicitly constrains the context digest γ , a proof created for one incident, protected-asset reference, destination, nonce, or expiry is not valid for a different public instance unless a new valid proof is produced. For operational deduplication, define

$$\text{claimID} = H_{\text{id}}(\text{Z-SCAPE/claim-id/v2} \parallel \text{Encode}(W, C, \text{incident}, \text{assetRef}, D_{\text{dest}}, n, \text{expiry})).$$

The verifier records consumed nonces and claim identifiers. Where the destination type supports an independent proof of control, the recovery procedure additionally verifies control of D_{dest} using the new wallet or account rather than the compromised key.

3.7 SNARK and STARK Proofs

Z-SCAPE allows different proof systems to implement the same credential relation.

- **PLONK-family zk-SNARK profile.** PLONK provides succinct proofs and verification with an updatable universal structured reference string [26]. This reduces proof size and verifier cost and is therefore suitable for constrained or on-chain verification. The original pairing-based setting is classified as a classical-security profile.
- **zk-STARK profile.** STARKs avoid a structured reference string and can rely on hash-function assumptions [27]. They provide a transparent alternative but generally require larger proofs and more verification bandwidth.

A change of proof system does not change W , P , r , or C provided the new circuit proves the same relation for the active hash profile. Changing the commitment hash from h_i to h_{i+1} , however, requires an authenticated update to a new commitment. A deployment claiming post-quantum protection must select a transparent proof system and symmetric/hash parameters whose concrete post-quantum security meets the intended level, and the use of Poseidon alone does not imply post-quantum security.

3.8 Credential updates and profile migration

Let (C_i, h_i, pol_i) be the active credential with sequence number i . A proposed transition is the public object

$$T_i = \text{Encode}(W, C_i, h_i, C_{i+1}, h_{i+1}, \text{pol}_{i+1}, i + 1, \text{action}).$$

The old-holder proof has public instance $x_i^{\text{old}} = T_i$ and witness (P_i, r_i) and proves that (P_i, r_i) opens (C_i, h_i, pol_i) . For ROTATE, TRANSFER, or MIGRATE, activation of C_{i+1} also requires a proof for the new credential whose witness (P_{i+1}, r_{i+1}) opens the proposed new commitment under h_{i+1} . The successor secret should be freshly generated for rotation or profile migration unless policy explicitly preserves the old secret (note that preserving an old r across rotations does not remediate a weak-RNG defect affecting r), and changes to the home address or security-question set likewise require a normal authenticated rotation.

The ledger adapter additionally requires current wallet authorization, except where a pre-enrolled threshold, successor, inheritance, or delayed-recovery policy explicitly defines a different rule. Requiring both current-wallet authorization and proof of the active recovery credential prevents an adversary holding only the reconstructed spending key from replacing the active commitment. Compromise of only one recovery factor is not sufficient to produce the credential-transition proof. If the complete recovery witness (P_i, r_i) is compromised but the wallet key remains secure, the normal transition rule still blocks state modification, and if both wallet authorization and the complete recovery witness are compromised, credential-update protection is lost.

4 Instantiations for Bitcoin and Ethereum

4.1 Bitcoin: Output-descriptor-scoped UTXO Commitment Recording

For Bitcoin, D contains the network identifier, a protocol-versioned descriptor byte string, the permitted derivation-branch semantics, and the descriptor-defined derivation domain [3]. Z-SCAPE does not claim that BIP-380 itself supplies a unique canonical serialization. Instead, the adapter defines $\text{Encode}_{\text{BTC}}(D)$ explicitly, including descriptor checksum handling, key-origin notation, wildcard semantics, and network identifier. The public Z-SCAPE record carries the stable wallet identifier W , the credential commitment C , and protocol metadata. Disclosure of an extended-public-key descriptor is not required on-chain.

For an enrolment transaction tx , $\text{VerifyBinding}_{\text{BTC}}(D, tx) = 1$ only if the prover provides a descriptor-membership witness μ (for example the derivation branch and index such that deterministic descriptor evaluation at μ gives the locking script of at least one spent prevout). This avoids modeling the possibly unbounded descriptor language as a set that must be enumerated. The verifier also checks $H_{\text{id}}(\text{Z-SCAPE/wallet/v3} \parallel \text{Encode}_{\text{BTC}}(D)) = W$.

A compact Z-SCAPE data-carrier format encodes the seven-byte ASCII tag Z-SCAPE, one-byte protocol version, one-byte action, one-byte hash-configuration identifier, a four-byte sequence number, the 32-byte stable wallet identifier W , and the 32-byte credential commitment C , for a 78-byte payload. Including W explicitly allows indexers and verifiers to associate a

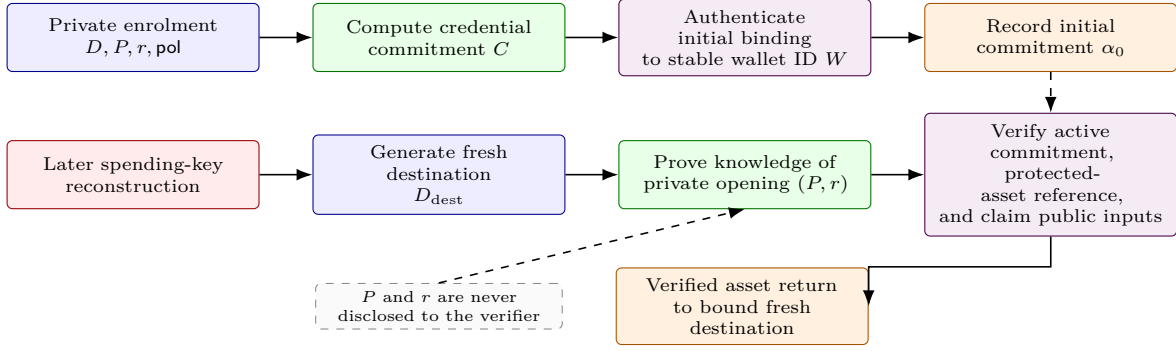


Figure 2: Z-SCAPE enrolment and recovery proof. The ledger records the stable wallet identifier W , credential commitment C , and protocol metadata, while the private opening (P, r) remains off-chain. After compromise of the wallet signing key, the prover proves knowledge of both recovery factors in ZK. The verifier independently resolves the authenticated protected-asset reference for $(\text{incident}, W)$, and the proof binds that reference together with the incident, fresh destination, nonce, and expiry.

record with its wallet context without first recovering the descriptor or other off-chain wallet metadata. Implementations must treat node relay and miner data-carrier limits as deployment policy rather than consensus rules. The descriptor, derivation witness, mandatory personal record, and recovery secret remain off-chain.

Bitcoin does not provide a native Z-SCAPE state machine. Verifiers therefore reconstruct the active commitment from finalized Z-SCAPE records and authenticated transition packages. For a record α , define

$$\text{order}_{\text{BTC}}(\alpha) = (h_\alpha, \text{txindex}_\alpha, \text{vout}_\alpha),$$

where h_α is the block height, txindex_α is the transaction index within the block, and vout_α is the Z-SCAPE output index. Eligible records are ordered lexicographically only after the adapter-specific finality policy is met. The initial commitment is the earliest eligible ENROLL record under this order. A later ROTATE, TRANSFER, MIGRATE, or REVOKE record is accepted only if its transition package contains the required proof of the active credential and the required wallet-authorization evidence.

A post-enrolment attacker who reconstructs only the Bitcoin spending key can create new transactions and additional OP_RETURN records, but cannot produce a valid transition proof for the active recovery credential. At claim time, verification is normally off-chain, and Bitcoin consensus need not understand PLONK, STARK, or Poseidon. The recovery process must separately verify control of the fresh Bitcoin destination using BIP-322 signed-message proofs where supported, or another destination-type-specific proof-of-control procedure defined by the adapter [4]. Protective transfer still operates over discrete UTXOs and must satisfy each original locking script. Z-SCAPE supplies a recovery proof, and it does not add a spending condition to existing outputs.

Descriptor disclosure can itself be privacy-sensitive. An extended public key may reveal a broad set of derived addresses to the verifier. Deployments should therefore limit enrolment to the required derivation range, avoid publishing descriptors on-chain, and may use a dedicated enrolment branch or a separately authenticated key where appropriate.

4.2 Ethereum: account semantics and registry state

Ethereum requires a different adapter because account authority can arise from several account configurations. Z-SCAPE distinguishes (i) a plain EOA whose native authority is its secp256k1 key, (ii) an EIP-7702 delegated EOA, which continues to use EOA key authority while executing code through a delegation designator, and (iii) a contract account whose authorization logic is defined by deployed code [29, 31]. The stable context

D_{ETH} contains at least chain ID, account type, canonical account address, registry instance, and credential domain. The account-type field distinguishes these cases, and the chain ID is essential because the same hexadecimal address can exist on multiple Ethereum-family networks [30].

For delegated EOAs and contract accounts, binding only the account address or a bytecode hash may not be sufficient. An EIP-7702 delegated EOA can change or clear its delegation, while an upgradeable contract wallet can keep the same proxy address or proxy code while implementation addresses, owner sets, recovery-authorizer sets, thresholds, modules, or recovery policies change. The adapter therefore computes a versioned authorization-configuration digest

$$Q_i = H_{\text{id}}\left(\text{Encode}(\text{delegation}_i, \text{implementation}_i, \text{owners}_i, \text{recoveryAuthorizers}_i, \text{threshold}_i, \text{modules}_i, \text{recoveryPolicy}_i)\right).$$

with fields selected according to the account architecture, and fields that are not applicable to a given account type are encoded using configuration-defined null values. For an EIP-7702 delegated EOA, delegation_i records the active delegation target or the configuration-defined null value if no delegation is active. Q_i is incorporated into pol_i rather than the stable wallet context so that authorization-configuration changes do not change W . A dedicated registry contract maintains the active commitment, hash configuration, policy digest, and sequence number. Initial EOA enrolment requires a transaction from the bound account or equivalent authorization verified by the registry. Contract-account enrolment may be invoked by the account or authenticated through ERC-1271 validation [32]. Rotation, transfer, revocation, and hash-configuration migration require current account authorization plus acceptance of the credential-transition proof. Z-SCAPE defines two verification modes with different trust assumptions. In on-chain mode, a verifier contract checks the selected proof system and the registry updates state only after successful verification. In threshold-authorized off-chain mode, the proof is checked off-chain by an m -of- n verifier committee and the registry accepts only a threshold signature over a canonical transcript containing W , the old and new commitments, the commitment-hash configuration identifiers, the proof system identifier, sequence number, action, and proof digest. Credential-state integrity in this mode is conditional on the threshold assumption and the committee's key security. A single authorized verifier is not sufficient for the credential-state-integrity guarantee defined by the protocol. For a compromised plain or EIP-7702 delegated EOA, key reconstruction exposes not only ETH but also token balances, NFTs, allowances, staking positions, bridge claims, governance rights, and application state. Asset return therefore requires an asset inventory and protocol-specific calls. The fresh destination should be independently

generated, and control must be verified by an EOA signature or, for a contract destination, the account's deployed validation logic such as ERC-1271. If a contract account already implements an independent recovery-authorizer set, threshold authorization scheme, timelock, or other recovery module, that native recovery path should normally be preferred.

5 Recovery-Credential Custody and Asset Return Protocol

5.1 Recovery Credential Record and factor separation

Z-SCAPE defines a private Recovery Credential Record (RCR) containing the mandatory personal record P , the 256-bit recovery secret r , wallet context D , active policy pol , active commitment-record reference loc_α , hash-profile identifier, and credential-update packages. Knowledge of the exact pair (P, r) is required to generate a valid recovery proof. The RCR must preserve the canonical enrolment-time home address and all mandatory question identifiers and answers exactly. Because address and answer normalization errors can cause permanent recovery failure, wallet software should display an enrolment transcript and permit an offline verification test before final commitment recording. Additional purchase records, identity documents, funding records, or inherited-rights evidence may still be stored separately as supporting material, but they are outside the base ZK relation.

The two factors should be stored and generated through different failure domains. The recovery secret must not be derived from the wallet seed or intentionally generated through the same RNG path. The personal record should not be stored only inside the wallet device or the same seed backup. Deployments may store r in a separate hardware device, split it using a threshold secret-sharing process, or hold it in geographically separated encrypted backups, while preserving P through a separately protected human-readable or encrypted record. A deployment may reconstruct threshold shares only inside the proving environment. The custody mechanism is outside the proof relation but is part of the operational threat model.

5.2 Pre-authorized protective-transfer and treasury redistribution protocol

Assume that a manufacturer or independent incident-response team confirms a systemic entropy-source failure and can reconstruct affected keys before coordinated disclosure. Immediate disclosure risks a generalized race to drain exposed assets. Under a valid operational mandate, the response entity executes an advance protective sweep into a protected, threshold-controlled recovery treasury. To return these secured assets without compromising user privacy or relying on exposed signing keys, the treasury operator acts as a verifier $\mathcal{T}$ for Z-SCAPE recovery claims. We use the COLDCARD incident as motivation for this hypothetical protocol, and the steps below do not represent Coinkite's actual incident response. The protocol proceeds as follows:

Protocol 1. Z-SCAPE Emergency Recovery

Require: Incident identifier incident ; affected wallet set $\mathcal{W}$; active recovery records; authenticated protective-transfer records; ledger adapter $\mathcal{A}$; recovery policy pol ; proof system z with parameters pp_z and verifier key vk_z

Ensure: Only assets authenticated as protectively transferred from wallet W under incident are returned to the verified fresh destination bound to an accepted recovery proof

- 1: Obtain independent confirmation that the reported vulnerability permits practical reconstruction of affected wallet keys.

- 2: Determine the affected outputs, accounts, authorization paths, and asset classes.
- 3: **if** the vulnerability is not independently confirmed **then**
- 4: **return** $\perp$
- 5: **end if**
- 6: Identify the exposed assets eligible for recovery under pol .
- 7: Transfer only those assets to a protected, threshold-controlled recovery treasury.
- 8: Distribute the threshold authorization shares among independent recovery operators according to the applicable governance policy.
- 9: **for** each affected wallet identifier $W \in \mathcal{W}$ **do**
- 10: Determine the exact source assets originating from W that were included in the protective transfer.
- 11: Construct an authenticated protected-asset record containing the canonical source references sources , corresponding treasury-transfer references treasuryRefs , and canonical asset inventory assetSet .
- 12: Compute assetRef according to Equation (1).
- 13: Associate the authenticated assetRef with $(\text{incident}, W)$.
- 14: **end for**
- 15: Publish the incident information, affected wallet or software versions, claim procedure, eligibility rules, and audit requirements.
- 16: **for** each recovery claim for an affected wallet identifier W **do**
- 17: Resolve the active authenticated (C, h, pol) for W under the ledger adapter's finality and credential-update rules.
- 18: **if** no valid active commitment exists for W **then**
- 19: Reject the claim.
- 20: **continue**
- 21: **end if**
- 22: Resolve the authenticated protected-asset record for $(\text{incident}, W)$ and obtain assetRef .
- 23: **if** no valid protected-asset record exists for $(\text{incident}, W)$ **then**
- 24: Reject the claim.
- 25: **continue**
- 26: **end if**
- 27: Generate a fresh destination D_{dest} using an entropy source independent of the compromised wallet.
- 28: Obtain a fresh verifier nonce $n \leftarrow \{0, 1\}^{256}$ and claim expiry value expiry .
- 29: Compute the public claim-context digest γ using Equation (5).
- 30: Construct the public claim instance:

$$x_{\text{claim}} = (W, C, h, \text{pol}, \text{incident}, \text{assetRef}, D_{\text{dest}}, n, \text{expiry}, \gamma).$$
- 31: Generate the ZK recovery proof:

$$\pi_{\text{claim}} \leftarrow \text{Prove}_z(pp_z, x_{\text{claim}}; P, r).$$
- 32: Verify the recovery proof:

$$\text{Verify}_z(vk_z, x_{\text{claim}}, \pi_{\text{claim}}) = 1.$$
- 33: **if** proof verification fails **then**
- 34: Reject the claim.
- 35: **continue**
- 36: **end if**
- 37: Compute

$$\text{claimID} = H_{\text{id}}(\text{Z-SCAPE/claim-id/v2} \parallel \text{Encode}(W, C, \text{incident}, \text{assetRef}, D_{\text{dest}}, n, \text{expiry})).$$
- 38: **if** the claim has expired or n or claimID has already been consumed **then**
- 39: Reject the claim.
- 40: **continue**
- 41: **end if**
- 42: Verify control of D_{dest} independently of the compromised wallet key using the adapter-defined destination-control procedure. Bitcoin and Ethereum adapters must perform this check.
- 43: **if** destination-control verification fails **then**
- 44: Reject the claim.

```

45:   continue
46: end if
47: if additional legal or policy review is required then
48:   Validate the required external evidence under pol.
49:   if the additional review fails then
50:     Reject the claim.
51:   continue
52: end if
53: end if
54: Verify that the assets identified by assetRef have not
   already been returned under a previously completed claim.
55: if the protected assets have already been returned then
56:   Reject the claim.
57: continue
58: end if
59: Atomically authorize and record the return only of the as-
   sets identified by the authenticated assetRef for (incident, W)
   and only to the verified  $D_{\text{dest}}$  bound to the accepted proof
   instance; in the same state transition, mark  $n$  and claimID
   as consumed and mark the protected assets as returned.
   Concurrent claims must not be able to observe the same
   assetRef as unreturned and both complete.
60: Mark the claim as completed.
61: end for
62: return Complete

```

5.3 Claim verification

Claim verification proceeds as follows:

Protocol 2. Z-SCAPE Claim Verification

Require: Wallet context D ; Z-SCAPE enrolment-record ref-
erence loc_α ; incident identifier incident ; authenticated
protective-transfer records; proof system identifier z ; verifier
key vk_z

Ensure: A claim is accepted only if it opens the active pre-
enrolled credential for W and authorizes only the assets
authenticated as protectively transferred from that same W
under incident to the destination bound to the proof

```

1: Resolve  $D$  and  $\text{loc}_\alpha$  and obtain the stable wallet identifier
    $W$ .
2: Reconstruct the active  $(C, h, \text{pol})$  under the ledger-specific
   finality and credential-update rules.
3: if the commitment configuration  $h$  or proof system  $z$  is
   unsupported or deprecated then
4:   Reject the claim.
5:   return  $\perp$ 
6: end if
7: Resolve the authenticated protected-asset record for
   (incident,  $W$ ).
8: if no valid protected-asset record exists for (incident,  $W$ )
   then
9:   Reject the claim.
10:  return  $\perp$ 
11: end if
12: Recompute assetRef from the authenticated sources,
   treasuryRefs, and assetSet according to Equation (1).
13: Verify that the protected-asset record associates assetRef
   with the same (incident,  $W$ ).
14: if the protected-asset binding is invalid then
15:   Reject the claim.
16:   return  $\perp$ 
17: end if
18: Receive the proposed fresh destination  $D_{\text{dest}}$ .
19: Sample a fresh verifier nonce  $n \leftarrow \{0, 1\}^{256}$  and set the claim
   expiry.
20: Compute the public claim-context digest  $\gamma$  using Equa-
   tion (5).
21: Construct the public claim instance:
    $x_{\text{claim}} = (W, C, h, \text{pol}, \text{incident}, \text{assetRef}, D_{\text{dest}}, n, \text{expiry}, \gamma)$ .

```

```

22: Provide  $x_{\text{claim}}$  to the prover.
23: Receive the recovery proof  $\pi_{\text{claim}}$  from the prover.
24: Compute
   claimID =  $H_{\text{id}}(\text{Z-SCAPE/claim-id/v2} \parallel$ 
   Encode( $W, C, \text{incident}, \text{assetRef}, D_{\text{dest}}, n, \text{expiry}$ )).
25: if the claim has expired then
26:   Reject the claim.
27:   return  $\perp$ 
28: end if
29: if  $n$  or claimID has already been consumed then
30:   Reject the claim.
31:   return  $\perp$ 
32: end if
33: Verify the recovery proof:
   Verifyz( $vk_z, x_{\text{claim}}, \pi_{\text{claim}}$ ) = 1.
34: if proof verification fails then
35:   Reject the claim.
36:   return  $\perp$ 
37: end if
38: Verify control of  $D_{\text{dest}}$  independently of the compromised
   wallet key using the adapter-defined destination-control pro-
   cedure. Bitcoin and Ethereum adapters must perform this
   check.
39: if destination-control verification fails then
40:   Reject the claim.
41:   return  $\perp$ 
42: end if
43: Perform any required sanctions, inheritance, identity, asset-
   origin, competing-claim, or other policy checks outside the
   base ZK relation.
44: if any required external check fails then
45:   Reject the claim.
46:   return  $\perp$ 
47: end if
48: Verify that the assets identified by assetRef have not already
   been returned under a previously completed claim.
49: if the protected assets have already been returned then
50:   Reject the claim.
51:   return  $\perp$ 
52: end if
53: Atomically approve and record the return only of the assets
   identified by assetRef for (incident,  $W$ ) and only to the exact
    $D_{\text{dest}}$  bound to the accepted proof instance; in the same
   state transition, mark  $n$  and claimID as consumed and mark
   the assets as returned. Concurrent claims must not both
   complete against the same unreturned assetRef.
54: return Accept

```

For any implementation in which multiple verifiers or treasury operators can process claims concurrently, the state transition that consumes n and claimID and marks assetRef as returned must be serialized or otherwise atomic. Proof verification may occur in parallel, but two claims must not both commit a successful return against the same unreturned protected asset record.

A compromised spending or owner key is not accepted as the sole recovery authentication witness because both the legitimate controller and an adversary that reconstructs the key can satisfy the same native key-based authorization rule. Z-SCAPE instead verifies knowledge of the independently enrolled recovery credential (P, r) .

6 Implementation and Evaluation Plan

The present work specifies the protocol and ledger adapters but does not report a production implementation or measured performance results. We therefore make no empirical claim about prover latency, proof size, Ethereum gas cost, or end-to-end recovery throughput. A conforming evaluation should implement at least one PLONK-family proof system and one transparent

STARK proof system and report the following quantities under fixed hardware, software, and security parameters:

- circuit or AIR size for canonical P encoding, the active credential hash, and public-input binding,
- prover time, peak memory, verifier time, and proof size,
- the effect of Poseidon versus SHA-256 on constraint count and prover cost,
- Bitcoin commitment-record payload size, transaction weight contribution, and fee under stated fee-rate assumptions,
- Ethereum gas for enrolment, transition, and on-chain proof verification, or threshold-signature verification in the off-chain mode,
- test vectors for canonical home-address/question encoding, byte-to-field chunking, wallet-context hashing, claim identifiers, commitment-hash migration, and proof system migration, and
- negative tests covering replay, wrong-destination substitution, cross-wallet protected-asset substitution, claimant-supplied asset-reference substitution, stale sequence numbers, invalid descriptor membership, and proof system or hash-configuration downgrade attempts.

Until these measurements are reported, Z-SCAPE should be evaluated as a protocol design and security model rather than as a demonstrated high-performance recovery system.

7 Security Model and Informal Analysis

7.1 Adversarial view and security goals

Let V_A contain the complete public ledger, the wallet context discoverable from public or subsequently disclosed data, all public Z-SCAPE metadata, any spending seed or owner key exposed after the first valid enrolment, and arbitrary auxiliary information relevant to P . This view includes any wallet-to-person linkage available through blockchain analysis, KYC or exchange records, public disclosures, device records, breached data, OSINT, or social knowledge. Z-SCAPE does not assume that the wallet controller remains anonymous. Rather, the uncertainty of P is evaluated conditional on whatever identity information is actually present in V_A . Unless a separate compromise is modeled, the exact private opening (P, r) and uncompromised threshold shares of r are excluded.

Z-SCAPE targets seven cryptographic properties. The analysis below is assumption-based, and formal reductions and concrete-security bounds depend on the selected commitment hash and proof system instantiation.

1. *Credential binding*: an active commitment should not admit a feasible alternative opening (P', r') for the same wallet and policy,
2. *Offline guessing resistance*: the public commitment should not permit practical recovery of the exact pair (P, r) under the stated assumptions about the recovery-secret source and the adversary’s auxiliary information,
3. *Security under reduced recovery-secret entropy*: if the conditional min-entropy of r is lower than intended, the attacker must still recover or guess the unknown components of the RNG-independent factor P ,
4. *Claim knowledge soundness*: an accepted proof should imply knowledge of a valid opening under the selected proof system assumptions,

5. *Claim privacy*: the proof should reveal no information about P or r beyond the truth of the proved relation and explicitly public values, and
6. *Credential-state integrity*: compromise of the native wallet key after enrolment should not permit replacement of the active credential without satisfying the configured update rule.
7. *Protected-asset scope integrity*: an accepted recovery claim for wallet W must authorize only assets whose authenticated protective-transfer record associates them with the same (incident, W).

7.2 Credential binding and offline guessing resistance

For fixed W , h , and pol ,

$$C = H_h(\text{Z-SCAPE/credential/v6} \parallel W \parallel \text{ZKEncode}(P) \parallel r \parallel H_{\text{id}}(\text{Encode}(\text{pol}))).$$

Binding to one enrolled credential relies on second-preimage resistance for a known valid opening and, more generally, collision resistance of the selected hash together with unambiguous encoding. Under normal operation, r is assumed to be generated from an entropy source independent of the signing seed, with a 256-bit target size and sufficient conditional min-entropy, so exhaustive recovery of the opening is dominated by the search for r . The mandatory record P is included as an RNG-independent factor rather than for a claimed fixed entropy contribution.

Z-SCAPE also models a degraded-entropy scenario. Rather than identifying a single shorthand value $H_\infty(r \mid V_A)$ with all more strongly conditioned cases, we define the residual recovery-secret parameter through the pointwise guessing bound actually required by the opening analysis. For every feasible personal record p under adversarial view $V_A = v$, assume

$$\max_{\rho} \Pr[r = \rho \mid P = p, V_A = v] \leq 2^{-t}.$$

The relevant quantity for recovery of the complete opening is the joint conditional distribution of (P, r) given V_A . We therefore do not add standalone quantities such as $H_\infty(P) + H_\infty(r)$, because the two factors may be correlated through the adversary’s side information and the distribution of human-derived data is highly non-uniform. The attacker’s uncertainty about P also depends on whether the wallet can be linked to a specific person. Z-SCAPE publishes neither a user identity nor the personal fields in P . If V_A does not contain a reliable wallet-to-person link, the attacker must additionally identify candidate personal profiles before efficiently testing candidate records. If blockchain analysis, KYC or exchange records, public disclosures, breached data, or other sources reveal the wallet controller, the candidate space for P may shrink substantially. Z-SCAPE therefore treats this unlinkability as a possible source of residual uncertainty, not as a guaranteed number of security bits. When r is predictable and all of P is known or efficiently enumerable, the commitment can become guessable. Conversely, when r has reduced entropy but some components of P remain unknown, those components add a second search dimension that does not depend on the failed RNG. For computational hiding or pseudorandomness of C with respect to low-entropy components of P , an explicit hiding assumption is still required (for example, a random-oracle or pseudorandomness model), and collision resistance alone is not a hiding assumption.

7.3 Quantitative bound for the degraded-entropy case

We now formalize the calculation in Section 3.4. The bound is stated directly for the committed opening (P, r) ; a wallet-

controller identity is not included as a separate factor because it is not part of the commitment opening. Let

$$P = (X_1, \dots, X_\ell)$$

and let $V_A = v$ denote a fixed adversarial view. Suppose that the following pointwise bounds hold for the threat model being evaluated.

First, for each personal-record component X_j and every feasible preceding history,

$$\max_x \Pr[X_j = x \mid X_{<j} = x_{<j}, V_A = v] \leq 2^{-s_j}, \quad (7)$$

where $X_{<j} = (X_1, \dots, X_{j-1})$.

Second, after the entropy-source defect and all auxiliary information have been taken into account, suppose that for every feasible p ,

$$\max_\rho \Pr[r = \rho \mid P = p, V_A = v] \leq 2^{-t}. \quad (8)$$

The parameter t may be zero in the worst case where r is completely predictable. This pointwise definition is intentionally stronger and more specific than informally writing $t = H_\infty(r \mid V_A)$, because conditioning additionally on P can change the attacker's guessing probability for r .

Proposition (sequential opening-guess bound). Under Equations (7) and (8), the probability of any particular committed opening

$$(P, r) = (p, \rho)$$

conditioned on $V_A = v$ is at most

$$2^{-B}, \quad B = \sum_{j=1}^{\ell} s_j + t. \quad (9)$$

Consequently, an adversary that tests at most Q distinct candidate openings has success probability at most

$$\Pr[\text{GuessOpen}_Q] \leq Q 2^{-B}, \quad (10)$$

apart from attacks that violate the assumed commitment-hash or proof system security.

Argument. By the probability chain rule,

$$\begin{aligned} \Pr[P = p, r = \rho \mid V_A = v] \\ &= \prod_{j=1}^{\ell} \Pr[X_j = x_j \mid X_{<j} = x_{<j}, V_A = v] \\ &\quad \cdot \Pr[r = \rho \mid P = p, V_A = v]. \end{aligned} \quad (11)$$

Applying Equations (7) and (8) gives

$$\begin{aligned} \Pr[P = p, r = \rho \mid V_A = v] &\leq \left(\prod_{j=1}^{\ell} 2^{-s_j} \right) 2^{-t} \\ &= 2^{-(\sum_j s_j + t)} = 2^{-B}. \end{aligned} \quad (12)$$

No statistical independence between the components of P is required: each bound s_j is already conditioned on all preceding components and on the adversary's auxiliary information. Likewise, Equation (8) conditions the residual unpredictability of r on the complete value of P .

For an adversary testing a set $\mathcal{G}$ of at most Q candidate openings, the union bound gives

$$\begin{aligned} \Pr[(P, r) \in \mathcal{G} \mid V_A = v] &\leq \sum_{g \in \mathcal{G}} \Pr[(P, r) = g \mid V_A = v] \\ &\leq Q 2^{-B}, \end{aligned} \quad (13)$$

which establishes Equation (10).

This formulation also explains why wallet-to-person unlinkability is not added as a separate entropy term. The public commitment is opened by (P, r) , not by an abstract identity variable. If identity information is unavailable, that fact may broaden the distribution of plausible personal records and therefore increase some calibrated s_j values. If KYC records, blockchain analysis, public disclosure, or breached data reveal the controller, those same s_j values may fall. Either way, the effect is incorporated into the conditional distribution of P under V_A rather than added independently.

A worked example. Applying the deliberately synthetic values of Table 1 gives

$$\sum_j s_j = 111.$$

If the recovery secret is completely predictable, $t = 0$, then

$$B = 111,$$

which is below a 128-bit target. If instead the recovery secret maintains the pointwise bound $t = 32$, then

$$B = 111 + 32 = 143.$$

Thus the example illustrates composition of conditional guessing bounds; it does not establish that human-derived questions provide 111 bits in any real deployment. Real values must be measured or conservatively justified under the actual adversarial view.

Interpretation and cryptographic ceiling. The quantity B is a bound on guessing the enrolled opening under a specified adversarial view. It is not by itself a universal claim of B -bit cryptographic security for the complete protocol. Overall concrete security is additionally limited by the selected commitment hash, the ZK proof system, its soundness parameters, implementation security, and the validity of the distributional assumptions used to obtain the s_j and t bounds. A useful summary is therefore

$$\lambda_{\text{eff}} \leq \min \{B, \lambda_H, \lambda_{\text{ZK}}\}, \quad (14)$$

where λ_H denotes the relevant concrete security level of the commitment-hash configuration and λ_{ZK} denotes the concrete security level of the selected proof system profile. Accordingly, the synthetic $B = 143$ example should be read only as an illustration of how deployment-specific pointwise bounds compose. It is not a claim that security questions inherently provide a fixed amount of entropy, and it provides no security guarantee unless the assumed conditional guessing bounds are justified for the deployment.

7.4 Knowledge soundness and ZK

An accepted claim requires

$$\text{Verify}_z(vk_z, x_{\text{claim}}, \pi_{\text{claim}}) = 1.$$

Under the knowledge-soundness assumption of the selected proof system, an adversary that produces an accepting proof for the active commitment must possess a witness (P, r) satisfying Equation (6), except with negligible probability or by violating the proof system assumptions. Under ZK, the proof does not disclose the home address, security-question answers, or r .

This argument is proof system dependent. A PLONK-family proof system inherits assumptions from its polynomial-commitment and setup construction, and the original PLONK design uses an updatable universal structured reference string [26]. A STARK proof system avoids a structured reference string and uses a transparent, hash-based approach [27]. The Z-SCAPE record stores the commitment-hash configuration identifier, while the verifier stores the circuit identifier, verifier-key or parameter digest, and proof system version used for each accepted claim.

7.5 Replay and destination redirection

The incident identifier, protected-asset reference, destination, nonce, and expiry are elements of x_{claim} . Changing any of them changes the statement supplied to Verify_z . A transcript accepted for one public instance is therefore not an accepting proof for another public instance unless the adversary produces a new valid proof under the selected proof system.

The verifier rejects reused nonces, consumed claim identifiers, and expired claims. Destination-control verification further protects against typographical errors, substituted addresses before proof generation, and claims that bind funds to a destination the user does not control.

7.6 Protected-asset integrity

The recovery credential is enrolled for wallet identifier W before compromise, whereas the protected-asset record is constructed after an incident from the actual protective transfers. These bindings serve different purposes. The pre-enrolled commitment prevents an adversary who reconstructs the native wallet key after compromise from registering a replacement recovery credential for W . The authenticated protected-asset record determines which assets were actually transferred from W into the recovery treasury.

For each claim, the verifier independently resolves `assetRef` from the protective-transfer records for $(\text{incident}, W)$ and does not accept an arbitrary asset set supplied by the claimant. Because W and `assetRef` are elements of x_{claim} , replacing either with the identifier or protected-asset reference of another wallet changes the proved public instance. Consequently, a valid proof for wallet W_1 does not authorize assets whose authenticated protective-transfer record is associated with wallet W_2 .

7.7 Post-enrolment wallet-key compromise

After exposure of the native spending authority, $\mathcal{A}$ can authorize ledger transactions and may publish additional well-formed Z-SCAPE enrolment records. This does not change the active credential. The initial commitment is the unique enrolment selected or enforced by the ledger adapter for W , and every accepted replacement must satisfy the configured credential-update proof plus the required wallet authorization. Thus, the verifier need not know the exact time at which the adversary reconstructed the spending key.

If the wallet key was already compromised before the first valid enrolment, an adversary may front-run, censor, or create a competing authenticated initial enrolment. On Bitcoin, front-running means broadcasting a competing `OP_RETURN` Z-SCAPE record before the legitimate user's transaction is mined, potentially exploiting mempool visibility, transaction-fee bidding, or miner collaboration. On Ethereum, front-running may additionally exploit the public mempool and MEV (maximal extractable value) infrastructure to insert a competing registry call before the legitimate enrolment transaction. Z-SCAPE provides no recovery-authentication guarantee in that case unless initial setup uses an independent device-production or initial-setup attestation, or another independent trust anchor not controlled by the vulnerable key.

7.8 Recovery-witness compromise and loss

If only r is compromised while P remains unknown, the adversary still lacks the complete recovery witness and cannot generate an accepted Z-SCAPE claim. If P is exposed but r remains strong, the random factor continues to protect the opening. If both P and r are compromised, the adversary can generate recovery proofs, although a normal credential update still additionally requires current wallet authorization unless policy states otherwise. If the wallet key and the complete recovery

witness are compromised, the credential-update protection is lost.

Loss of either factor can be an availability failure because the ZK proof requires the exact committed opening. Deployments requiring resilience to loss should use separately protected backups of P , threshold storage of r , pre-enrolled successor commitments, inheritance policies, or delayed recovery. Allowing replacement from the compromised wallet key alone would reintroduce the post-compromise enrolment attack.

7.9 Cryptographic agility and migration

Proof-system agility and hash agility are handled differently. A new proof system may verify the same credential relation without changing C , provided its circuit implements the same active commitment-hash configuration and canonical encoding. A change of commitment hash requires an authenticated `MIGRATE` transition to a new commitment, with proof of the old opening and proof of knowledge of the new opening. This prevents a verifier from silently reinterpreting an existing credential under different hash semantics.

Poseidon parameters must be fixed and versioned, including the exact byte-to-field encoding. SHA-256 remains an interoperability option where prover cost is acceptable. Z-SCAPE does not equate “STARK” with unconditional post-quantum security. Concrete post-quantum security depends on the hash, query complexity, soundness parameters, and implementation choices [27]. The pairing-based PLONK instantiation is classified as classical unless a post-quantum polynomial-commitment construction is explicitly selected and analysed.

7.10 Verifier compromise and implementation leakage

ZK protects the witness from a protocol-conforming verifier but does not make the claim-verification infrastructure trustworthy. In Bitcoin's off-chain verification path, operational decisions should use threshold approval or independent review. In Ethereum's threshold-authorized off-chain mode, credential-state integrity is conditional on fewer than the threshold number of verifier keys being compromised. Formally, the assumption is that strictly fewer than k of n verifier committee members are corrupted, where k is the acceptance threshold. Compromise of k or more members allows the committee to accept fraudulent claims or block legitimate ones. On-chain verification removes this committee trust for proof validity but not for legal or operational decisions.

Implementations must also prevent recovery-witness leakage outside the proof system. Logging the home address or security answers, storing P or r in circuit witness files or swap, browser telemetry, crash dumps, or insecure remote proving can defeat the privacy objective even when the underlying proof system is ZK. Side-channel risks include timing variations in proof generation that may leak information about the witness structure, power-analysis attacks on constrained proving hardware, and cache-timing attacks on hash computations over P or r . Implementations should use constant-time cryptographic operations where feasible and avoid branching on secret-dependent values during proof computation.

7.11 Privacy and linkability

The public ledger reveals participation in Z-SCAPE because an enrolment record is associated with a stable wallet identifier. Neither P nor r appears on-chain. Under the stated preimage and, where claimed, hiding assumptions, the commitment does not reveal the private opening, and the ZK claim does not disclose the enrolment-time address or security-question answers. Repeated claims remain linkable to the same public commitment unless additional batching or anonymity techniques are used.

Merkle batching can reduce direct per-record linkage at the cost of inclusion-proof and archival requirements.

Z-SCAPE neither repairs a defective entropy source nor reverses an adversarial transfer that has achieved ledger finality. Entropy-source validation, fail-closed RNG integration, secure backup, threshold authorization, contract-native recovery, and rapid migration remain primary controls. Z-SCAPE supplies a recovery proof for the post-compromise setting. Knowledge of (P, r) is not legal identity or legal ownership. The combined recovery credential can be copied, transferred, stolen, inherited, or disclosed. A valid proof establishes knowledge of a pre-enrolled recovery credential bound to a fresh destination, and legal title remains an external question.

8 Discussion and Conclusion

Z-SCAPE provides cryptographic evidence for recovery after a signing key has ceased to uniquely identify the legitimate wallet controller. However, technical capability to reconstruct a vulnerable key and perform a protective transfer does not by itself establish legal authority to move the corresponding assets. Practical deployment therefore requires an appropriate authorization and governance model, such as pre-existing opt-in recovery policies, threshold control of recovery operations, and auditable asset-return procedures. The protocol minimizes disclosure of recovery information by keeping the personal record P and recovery secret r off-chain and proving knowledge of them in zero knowledge. Nevertheless, public wallet identifiers and commitments may remain linkable to wallet activity, and deployments must consider applicable privacy, custody, and regulatory requirements. These requirements remain external to the Z-SCAPE proof. In particular, a valid proof establishes knowledge of the pre-enrolled recovery credential; it does not by itself establish legal identity or legal ownership.

The COLDCARD incident demonstrates that the effective seed distribution is part of the wallet security boundary. If the root seed has not sufficient conditional min-entropy, deterministic derivation may yield signing keys that can be reconstructed without interacting with the wallet device. Once such a key is reconstructable, a valid native signature no longer uniquely identifies the legitimate controller. Z-SCAPE addresses this problem through a recovery credential enrolled before compromise and independent of the wallet's native signing authority. The credential combines a recovery secret r with an RNG-independent personal record P , and recovery is authorized through a zero-knowledge proof bound to the wallet identifier, incident, authenticated protected-asset reference, fresh destination, verifier nonce, and expiry. The verifier independently derives the protected-asset reference, ensuring that a valid proof can authorize only assets actually transferred from the corresponding wallet during the protective recovery process. Our proposal presents the protocol, security model, and Bitcoin and Ethereum integration mechanisms. It does not yet provide a production implementation or performance evaluation. Future work includes concrete PLONK and STARK implementations, formal security proofs, test vectors, wallet integration, usability studies, and measurements of Bitcoin transaction overhead and Ethereum verification costs.

Acknowledgments

The authors used AI tools only for grammar checking, language polishing, and improving readability. No AI system was used to generate the scientific contributions, experimental results, figures, or security claims presented in this paper. The authors reviewed and verified the entire manuscript and remain fully responsible for its correctness, originality, and integrity.

References

- [1] P. Wuille, “BIP-32: Hierarchical Deterministic Wallets”, Bitcoin Improvement Proposals, 2012.
<https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki>
- [2] M. Palatinus, P. Rusnak, A. Voisine, and S. Bowe, “BIP-39: Mnemonic code for generating deterministic keys”, Bitcoin Improvement Proposals, 2013.
<https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki>
- [3] P. Wuille and A. Chow, “BIP-380: Output Script Descriptors General Operation”, 2021.
<https://bips.dev/380/>
- [4] K.-J. Alm and O. Gugger, “BIP-322: Generic Signed Message Format”, Bitcoin Improvement Proposals, 2018.
<https://bips.dev/322/>
- [5] Coinkite, “Coldcard Security Advisory”, published 30 July 2026, updated 1 August 2026.
<https://blog.coinkite.com/coldcard-mk3-seed-generation-warning/>
- [6] Block Bitcoin Engineering and Security, with anonymous security researchers, “Predictable RNG Fallback and 32-Bit Reseed in COLDCARD Firmware”, published 30 July 2026, accessed 4 August 2026.
<https://engineering.block.xyz/blog/predictable-rng-fallback-and-32-bit-reseed-in-coldcard-firmware>
- [7] Coinkite, “Technical Deep Dive into the Entropy Issue”, published 30 July 2026, updated 1 August 2026.
<https://blog.coinkite.com/entropy-technical-backgrounder/>
- [8] S. Malwa, “How bitcoin cold wallets lost \$70 million in an attack that never touched the devices”, CoinDesk, 1 August 2026, accessed 4 August 2026.
<https://www.coindesk.com/tech/2026/08/01/how-bitcoin-cold-wallets-lost-usd70-million-in-an-attack-that-never-touched-the-devices>
- [9] S. Malwa, “Coldcard wallet losses may near \$114 million as possible fourth sweep emerges”, CoinDesk, 3 August 2026, accessed 4 August 2026.
<https://www.coindesk.com/tech/2026/08/03/coldcard-wallet-losses-may-near-usd114-million-as-possible-fourth-sweep-emerges>
- [10] O. Sattath and S. Wyborski, “Protecting Quantum Procrastinators with Signature Lifting: A Case Study in Cryptocurrencies”, Cryptology ePrint Archive, Paper 2023/362, 2023.
<https://eprint.iacr.org/2023/362>
- [11] J. Lopp, C. Papathanasiou, I. Smith, J. Ross, S. Vaile, and P.-L. Dallaire-Demers, “BIP-361: Post Quantum Migration and Legacy Signature Sunset”, Bitcoin Improvement Proposals, 2026.
<https://bips.dev/361/>
- [12] O. Osuntokun, “Post-Quantum BIP-86 Recovery via zk-STARK Proof of BIP-32 Seed Knowledge”, Bitcoin Development Mailing List, April 2026.
<https://groups.google.com/g/bitcoinddev/c/Q06piCEJhkI>

- [13] M. Sorf, P. Svenda, and L. Chmielewski, “Large-Scale Security Analysis of Hardware Wallets”, in *Availability, Reliability and Security (ARES 2025)*, Lecture Notes in Computer Science, vol. 15995, Springer, 2025.
https://doi.org/10.1007/978-3-032-00633-2_21
- [14] A. Scott and S. Andersen, “Engineering a backdoored bitcoin wallet”, in *18th USENIX WOOT Conference on Offensive Technologies (WOOT 24)*, pp. 89–100, 2024.
<https://www.usenix.org/conference/woot24/presentation/scott>
- [15] Ledger, “How Ledger Hardware Wallets (Signers) Work”, updated 19 June 2026, accessed 4 August 2026.
<https://www.ledger.com/academy/topics/ledgersolutions/how-ledger-hardware-wallets-work>
- [16] Trezor, “Trezor Safe 5: Secure Crypto Hardware Wallet”, accessed 4 August 2026.
<https://trezor.io/trezor-safe-5>
- [17] Trezor, “What is Shamir backup?”, Trezor Knowledge Base, accessed 4 August 2026.
<https://trezor.io/learn/advanced/standards-proposals/what-is-shamir-backup>
- [18] Coinkite, “COLDCARD Mk4: Secure Bitcoin Hardware Wallet”, accessed 4 August 2026.
<https://coldcard.com/mk4>
- [19] BitBox, “Security on every level: BitBox02 security features”, accessed 4 August 2026.
<https://bitbox.swiss/bitbox02/security-features/>
- [20] Blockstream, “Blockstream Jade: The Next-Generation Bitcoin Hardware Wallet”, accessed 4 August 2026.
<https://blockstream.com/jade/jade-plus/>
- [21] Keystone, “Keystone 3 Pro: Air-Gapped Crypto Hardware Wallet”, accessed 4 August 2026.
<https://keyst.one/shop/products/keystone-3-pro>
- [22] SafePal, “SafePal S1 Hardware Wallet”, accessed 4 August 2026.
<https://www.safepal.com/en/store/s1>
- [23] M. S. Turan, E. Barker, J. Kelsey, K. McKay, M. Baish, and M. Boyle, “Recommendation for the Entropy Sources Used for Random Bit Generation”, NIST SP 800-90B, 2018.
<https://doi.org/10.6028/NIST.SP.800-90B>
- [24] E. Barker, J. Kelsey, K. McKay, A. Roginsky, and M. S. Turan, “Recommendation for Random Bit Generator (RBG) Constructions”, NIST SP 800-90C, 2025.
<https://doi.org/10.6028/NIST.SP.800-90C>
- [25] D. Temoshok, J. Fenton, Y.-Y. Choong, N. Lefkovitz, A. Regenscheid, R. Galluzzo, and J. Richer, “Digital Identity Guidelines: Authentication and Authenticator Management”, NIST SP 800-63B-4, 2025.
<https://doi.org/10.6028/NIST.SP.800-63B-4>
- [26] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, “PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge”, in *Cryptology ePrint Archive, Paper 2019/953*, 2019.
<https://eprint.iacr.org/2019/953>
- [27] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Scalable, transparent, and post-quantum secure computational integrity”, *Cryptology ePrint Archive, Paper 2018/046*, 2018.
<https://eprint.iacr.org/2018/046>
- [28] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, “Poseidon: A New Hash Function for zero knowledge Proof Systems”, in *30th USENIX Security Symposium*, pp. 519–535, 2021.
<https://www.usenix.org/conference/usenixsecurity21/presentation/grassi>
- [29] Ethereum.org, “Ethereum accounts”, updated 13 April 2026, accessed 4 August 2026.
<https://ethereum.org/developers/docs/accounts/>
- [30] V. Buterin, “EIP-155: Simple replay attack protection”, *Ethereum Improvement Proposals*, 2016.
<https://eips.ethereum.org/EIPS/eip-155>
- [31] V. Buterin, S. Wilson, A. Dietrichs, and lightclient, “EIP-7702: Set Code for EOAs”, *Ethereum Improvement Proposals*, 2024.
<https://eips.ethereum.org/EIPS/eip-7702>
- [32] F. Giordano, M. Condon, P. Castonguay, A. Bandiali, J. Izquierdo, and B. Masius, “ERC-1271: Standard Signature Validation Method for Contracts”, *Ethereum Improvement Proposals*, 2018.
<https://eips.ethereum.org/EIPS/eip-1271>
- [33] V. Buterin, Y. Weiss, D. Tirosh, S. Nacson, A. Forshtat, K. Gazso, and T. Hess, “ERC-4337: Account Abstraction Using Alt Mempool”, *Ethereum Improvement Proposals*, 2021.
<https://eips.ethereum.org/EIPS/eip-4337>
- [34] European Data Protection Board (EDPB), “Guidelines 02/2025 on processing of personal data through blockchain technologies”, final version adopted 7 July 2026.
https://www.edpb.europa.eu/documents/guideline/guidelines-on-processing-of-personal-data-through-blockchain-technologies_en
- [35] Financial Action Task Force (FATF), “Updated Guidance for a Risk-Based Approach to Virtual Assets and Virtual Asset Service Providers”, 28 October 2021.
<https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Guidance-rba-virtual-assets-2021.html>
- [36] European Union, “Regulation (EU) 2023/1114 of the European Parliament and the Council of 31 May 2023, on markets in crypto-assets, and amending Regulations (EU) No 1093/2010 and (EU) No 1095/2010 and Directives 2013/36/EU and (EU) 2019/1937”, 2023.
<https://eur-lex.europa.eu/eli/reg/2023/1114/oj/eng>
- [37] A. Shamir, “How to share a secret”, *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
<https://doi.org/10.1145/359168.359176>
- [38] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems”, in *Advances in Cryptology – CRYPTO ’86*, Lecture Notes in Computer Science, vol. 263, Springer, pp. 186–194, 1987.
https://doi.org/10.1007/3-540-47721-7_12
- [39] D. R. Stinson, *Cryptography: Theory and Practice*, 3rd ed., CRC Press, 2005.